ДЗ-1. Аналитическая модель стоимости CNN

Efficient Models, ИТМО 2026, Денисова Карина, GPU: Tesla T4

- Вход $B \times 3 \times S \times S$, FP32 (4 байта), S кратно 16
- Свертки: padding $= K/2$, bias нет. ReLU inplace, eval() + inference_model()
- 1 MAC = 2 FLOPs. Свертка: $2 \cdot B \cdot C_{out} \cdot H_{out} \cdot W_{out} \cdot C_{in} \cdot K^2$. Linear: $2 \cdot B \cdot in \cdot out$
- Memory = max torch.cuda.max_memory_allocated() за один проход.
- Bytes: каждое ядро 1 раз читает входы и веса и 1 раз пишет выход

Размеры по слоям

$$H_{out} = \lfloor (H + 2p - K)/S \rfloor + 1 \quad \text{при } p = K/2 \rightarrow H_{out} = H/S$$

Слой	Выход	Элем. / картинку
x	$3 \times S \times S$	$3S^2$
conv1 7×7 s2, 3→32	$32 \times S/2 \times S/2$	$a_1 = 8S^2$
maxpool 3×3 s2	$32 \times S/4 \times S/4$	$a_2 = 2S^2$
Conv2 5×5 32→64	$64 \times S/4 \times S/4$	$a_3 = 4S^2$
Conv3 3×3 s2, 64→128	$128 \times S/8 \times S/8$	$a_4 = 2S^2$
Conv4 1×1, 128→256	$256 \times S/8 \times S/8$	$a_5 = 4S^2$
Conv5 3×3 s2, 256→256	$256 \times S/16 \times S/16$	$a_6 = S^2$
Conv6 1×1 256→512	$512 \times S/16 \times S/16$	$a_7 = 2S^2$
GAP → fc1 → fc2	$512 \to 256 \to 100$	

Параметры: 4704 + 51200 + 73728 + 32768 + 589824 + 131072 + 131328 + 25700 = 1 040 324 параметра =

= 4 161 296 байт

FLOPs(S, B)

$$
\begin{aligned}
&\text{Conv 1}: && 2 \cdot B \cdot 32 \cdot (S/2)^2 \cdot 3 \cdot 49 && = 2352\ BS^2 \\
&\text{Conv 2}: && 2 \cdot B \cdot 64 \cdot (S/4)^2 \cdot 32 \cdot 25 && = 6400\ BS^2 \\
&\text{Conv 3}: && 2 \cdot B \cdot 128 \cdot (S/8)^2 \cdot 64 \cdot 9 && = 2304\ BS^2 \\
&\text{Conv 4}: && 2 \cdot B \cdot 256 \cdot (S/8)^2 \cdot 128 && = 1024\ BS^2 \\
&\text{Conv 5}: && 2 \cdot B \cdot 256 \cdot (S/16)^2 \cdot 256 \cdot 9 && = 4608\ BS^2 \\
&\text{Conv 6}: && 2 \cdot B \cdot 512 \cdot (S/16)^2 \cdot 256 && = 1024\ BS^2 \\
&\text{fc}: && 2B \cdot (512 \cdot 256 + 256 \cdot 100) && = 313\,344\ B
\end{aligned}
$$

$$\text{FLOPs}(S, B) = 17\,712 \cdot B \cdot S^2 + 313\,344 \cdot B$$

ReLU / maxpool / GAP $\simeq 39 \cdot BS^2$ (0,2%) - не учитыв.

Пример: $S = 224, B = 1 \rightarrow 0{,}889$ GFLOPs

Memory (S, B) - считается точечно

Почему не предсказывает: размеры всех тензоров известны заранее, порядок выделения / освобождения задаёт код (пр. Sequential). Пик = max суммы живых тензоров.

Живёт всегда: веса W, вход x, рабочая область cuBLAS (выделяется 1 раз при первом matmul, не зависит от S, B; измерено 1 раз).

Во время слоя живы его вход и выход. ReLU inplace - 0 байт. MaxPool на CUDA держит индексы int64 (8 байт) размером с выход.

Во время	живые активации, байт/картинку
conv 1	$a1 = 32S^2$
maxpool	**$a1 + a2 + idx = 32 + 8 + 16 = 56S^2$**
conv 2	$8 + 16 = 24S^2$
conv 3	$16 + 8 = 24S^2$
conv 4	$8 + 16 = 24S^2$
conv 5	$16 + 4 = 20S^2$
conv 6	$4 + 8 = 12S^2$

Пик всегда на maxpool

$$Memory = W + R + 12 \cdot BS^2 + 56 \cdot BS^2 = 4\,161\,536 + R + 68 \cdot B \cdot S^2$$

4 161 536 = веса, каждый тензор округлен аллокатором до 512 байт

$S = 512$, $B = 256 \rightarrow 4{,}26$ ГиБ $< 14{,}6$ ГиБ $\Rightarrow$ OOM нет (был бы при $B \approx 300$)

Не учтено: рабочая область cuDNN внутри сверток – её выбирает эвристика cuDNN, PyTorch её размер не сообщает

Bytes moved (DRAM)

Ядро	читает + пишет (·4 байта)	байт / BS^2
Conv 1 / relu 1	3+8 / 8+8	44 / 64
maxpool	8+2 + idx 4	56
conv 2 / relu 2	2+4 / 4+4	24 / 32
conv 3 / relu 3	4+2 / 2+2	24 / 16
conv 4 / relu 4	2+4 / 4+4	24 / 32
conv 5 / relu 5	4+1 / 1+1	20 / 8
conv 6 / relu 6	1+2 / 2+2	12 / 16
GAP	2	8

$Bytes(S,B) = 380 \cdot B \cdot S^2 + 4\,161\,296 + 8\,592 \cdot B$

Интенсивность $17\,712 / 380 \approx FLOP/Байт$; у ReLU, maxpool, GAP = 0 (memory-bound)

Latency (S, B, 0)

Допущения:

- Каждое из 17 ядер – roofline: $t_k = \max(FLOPs_k / P, Bytes_k / BW)$
- Ядра идут последовательно, не перекрываются.
- t_0 – постоянная задержка (запуск ядер, Python, синхронизация)
- P и BW одни на все слои.

$$Latency = t_0 + \Sigma_k \max(FLOPs_k / P, Bytes_k / BW), \theta = (t_0, P, BW)$$

Режимы: launch-bound (мало работы, время $\approx t_0$) → memory-bound (ReLU, maxpool, 1×1: Bytes/BW) → compute-bound (conv2, conv5: $FLOPs / P$, время $\propto BS^2$).

Подбор: $\min \Sigma (\log pred - \log meas)^2$ только на базовой сетке (63 точки); проверка на 63 точках со случайными S и B.

Ограничения: $P \le 8{,}14$ TFLOP/s, $BW \le 320$ ГБ/с (паспорт T4).

FLOPs и байты всех слоев $\propto BS^2$ => без границ P и BW не различимы (минимум $P = 13$ TFLOP/s > пика T4)

Energy (S, B, θ_E)

CPU тратит постоянную мощность, пока работает, + энергию на FLOP и на байт

$$Energy = p_static \cdot Latency + e_flop \cdot FLOPs + e_byte \cdot Bytes, \theta_E \ge 0$$

линейно по Q_E => взвешенный МНК (по относительной ошибке), на базовой сетке.
Замер: $E = (E_NVML(конец) - E_NVML(начало)) / N$, N проходов ≥ 2 с.

Результаты (Tesla T4, PyTorch 2.11, cuDNN 9.27)

R (cuBLAS)	9 568 256 Б = 9,125 МиБ
t0 / P / BW	0,465 мс / 4,46 TFLOP/s / 87 Гб/с
p_static / e_flop / e_byte	50,8 Вт / 6,0 пДж / FLOP / 0
MAPE latency (калибр. / вал)	9,5% / 7,8%
MAPE energy (калибр. / вал)	9,2% / 8,8%
FLOPs vs FlopCounterMode	совпало 4 из 4
Memory (точна ± 1 МиБ)	38 из 132; в 94 — + cuDNN workspace
OOM	нет; max 4,26 ГиБ, как предсказано

Выводы

- $P = 4{,}46$ TFLOP/s $\approx 87\%$ пика при реальной частоте ~ 1 ГГц; T4 упирается в 70 Вт и снижает частоту 1590 → 870–1050 МГц.
- Формулы ломаются, где cuDNN меняет алгоритм (B = 4, conv 2) и на картах 2×2 ($S \leq 48$, больше B).
- Энергия $\approx$ мощность × время; e_byte = 0, т.к. байты α FLOPs